

Lab 4

Computer Security

Basic Network Security (80 Pts + 30 Optional)

Sniffing & Spoofing (40)

Firewall Configuration (40)

Optional Firewall (30)

For each Task: You must have a **dedicated section named Task [X]** (where X is the corresponding task) in your report that contains: 1) Relevant code/ configuration snippets, 2) a brief description of what the code or configuration does 3) Screenshots showing the relevant part clearly when applicable. For part two (firewall) iptables configuration is to be treated as a code .
Submissions without the relevant code or explanations will receive partial credit

Finally, if you use AI tools, you still have to justify (and are responsible) for their output. There are two sections in this assignment:

- 1) Sniffing and Spoofing
- 2) Firewall Configuration
 - a) Tasks 3.B,4 and 5 are OPTIONAL.

Both Labs are based on corresponding SEED labs but we only study part of them. You still need to download the corresponding arm or intel version for the lab setup as usual.

PART 1 (40PTS)

Packet Sniffing and Spoofing Lab

Copyright © 2006 - 2020 by Wenliang Du.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. If you remix, transform, or build upon the material, this copyright notice must be left intact, or reproduced in a way that is reasonable to the medium in which the work is being re-published.

1 Overview

Packet sniffing and spoofing are two important concepts in network security; they are two major threats in network communication. Being able to understand these two threats is essential for understanding security measures in networking. There are many packet sniffing and spoofing tools, such as Wireshark, Tcpdump, Netwox, Scapy, etc. Some of these tools are widely used by security experts, as well as by attackers. Being able to use these tools is important for students, but what is more important for students in a network security course is to understand how these tools work, i.e., how packet sniffing and spoofing are implemented in software.

The objective of this lab is two-fold: learning to use the tools and understanding the technologies underlying these tools. For the second object, students will write simple sniffer and spoofing programs, and gain an in-depth understanding of the technical aspects of these programs. This lab covers the following topics:

- How the sniffing and spoofing work
- Packet sniffing using the `pcap` library and Scapy
- Packet spoofing using raw socket and Scapy
- Manipulating packets using Scapy

Readings and Videos. Detailed coverage of sniffing and spoofing can be found in the following:

- Chapter 15 of the SEED Book, *Computer & Internet Security: A Hands-on Approach*, 2nd Edition, by Wenliang Du. See details at <https://www.handsonsecurity.net>.
- Section 2 of the SEED Lecture, *Internet Security: A Hands-on Approach*, by Wenliang Du. See details at <https://www.handsonsecurity.net/video.html>.

Lab environment. This lab has been tested on the SEED Ubuntu 20.04 VM. You can download a pre-built image from the SEED website, and run the SEED VM on your own computer. However, most of the SEED labs can be conducted on the cloud, and you can follow our instruction to create a SEED VM on the cloud.

Environment Setup using Container

In this lab, we will use three machines that are connected to the same LAN. We can either use three VMs or three containers. Figure 1 depicts the lab environment setup using containers. We will do all the attacks on the attacker container, while using the other containers as the user machines.

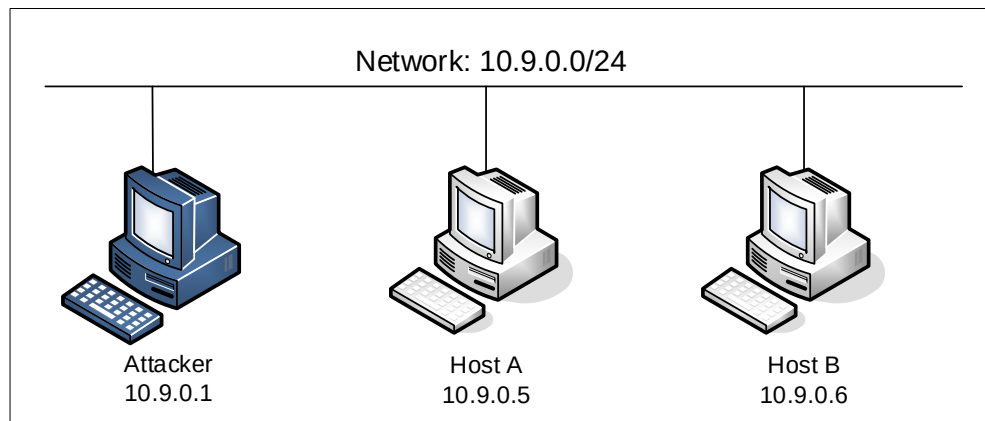


Figure 1: Lab environment setup

a. Container Setup and Commands

Please download the `Labsetup.zip` file to your VM from the lab's website, unzip it, enter the `Labsetup` folder, and use the `docker-compose.yml` file to set up the lab environment. Detailed explanation of the content in this file and all the involved `Dockerfile` can be found from the user manual, which is linked to the website of this lab. If this is the first time you set up a SEED lab environment using containers, it is very important that you read the user manual.

In the following, we list some of the commonly used commands related to Docker and Compose. Since we are going to use these commands very frequently, we have created aliases for them in the `.bashrc` file (in our provided SEEDUbuntu 20.04 VM).

```
$ docker-compose build # Build the container image
$ docker-compose up    # Start the container
$ docker-compose down  # Shut down the container

// Aliases for the Compose commands above
$ dcbuild              # Alias for: docker-compose build
$ dcup                 # Alias for: docker-compose up
$ dcdown               # Alias for: docker-compose down
```

All the containers will be running in the background. To run commands on a container, we often need to get a shell on that container. We first need to use the `"docker ps"` command to find out the ID of the container, and then use `"docker exec"` to start a shell on that container. We have created aliases for them in the `.bashrc` file.

```
$ dockps              // Alias for: docker ps --format "{{.ID}} {{.Names}}"
$ docksh <id>         // Alias for: docker exec -it <id> /bin/bash
```

```
// The following example shows how to get a shell inside hostC
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#

// Note: If a docker command requires a container ID, you do not need to
//       type the entire ID string. Typing the first few characters will
//       be sufficient, as long as they are unique among all the containers.
```

If you encounter problems when setting up the lab environment, please read the “Common Problems” section of the manual for potential solutions.

b. About the Attacker Container

In this lab, we can either use the VM or the attacker container as the attacker machine. If you look at the Docker Compose file, you will see that the attacker container is configured differently from the other containers. Here are the differences:

- *Shared folder.* When we use the attacker container to launch attacks, we need to put the attacking code inside the attacker container. Code editing is more convenient inside the VM than in containers, because we can use our favorite editors. In order for the VM and container to share files, we have created a shared folder between the VM and the container using the Docker `volumes`. If you look at the Docker Compose file, you will find out that we have added the following entry to some of the containers. It indicates mounting the `./volumes` folder on the host machine (i.e., the VM) to the `/volumes` folder inside the container. We will write our code in the `./volumes` folder (on the VM), so they can be used inside the containers.

```
volumes:
  - ./volumes:/volumes
```

- *Host mode.* In this lab, the attacker needs to be able to sniff packets, but running sniffer programs inside a container has problems, because a container is effectively attached to a virtual switch, so it can only see its own traffic, and it is never going to see the packets among other containers. To solve this problem, we use the `host` mode for the attacker container. This allows the attacker container to see all the traffics. The following entry used on the attacker container:

```
network_mode: host
```

When a container is in the `host` mode, it sees all the host’s network interfaces, and it even has the same IP addresses as the host. Basically, it is put in the same network namespace as the host VM. However, the container is still a separate machine, because its other namespaces are still different from the host.

Getting the network interface name. When we use the provided Compose file to create containers for this lab, a new network is created to connect the VM and the containers. The IP prefix for this network is

10.9.0.0/24, which is specified in the `docker-compose.yml` file. The IP address assigned to our VM is 10.9.0.1. We need to find the name of the corresponding network interface on our VM, because we need to use it in our programs. The interface name is the concatenation of `br-` and the ID of the network created by Docker. When we use `ifconfig` to list network interfaces, we will see quite a few. Look for the IP address 10.9.0.1.

```
$ ifconfig
br-c93733e9f913: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.1 netmask 255.255.255.0 broadcast 10.9.0.255
    ...
```

Another way to get the interface name is to use the `"docker network"` command to find out the network ID ourselves (the name of the network is `seed-net`):

```
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
a82477ae4e6b        bridge              bridge               local
e99b370eb525        host                host                 local
df62c6635eae        none                null                 local
c93733e9f913        seed-net            bridge               local
```

Lab Task Set 1: Using Scapy to Sniff and Spoof Packets

Many tools can be used to do sniffing and spoofing, but most of them only provide fixed functionalities. Scapy is different: it can be used not only as a tool, but also as a building block to construct other sniffing and spoofing tools, i.e., we can integrate the Scapy functionalities into our own program. In this set of tasks, we will use Scapy for each task.

To use Scapy, we can write a Python program, and then execute this program using Python. See the following example. We should run Python using the root privilege because the privilege is required for spoofing packets. At the beginning of the program (Line ①), we should import all Scapy's modules.

```
# view mycode.py
#!/usr/bin/env python3

from scapy.all import *    ①

a = IP()
a.show()

# python3 mycode.py
###[ IP ]###
    version    = 4
    ihl        = None
    ...

// Make mycode.py executable (another way to run python programs)
# chmod a+x mycode.py
# mycode.py
```

We can also get into the interactive mode of Python and then run our program one line at a time at the

Python prompt. This is more convenient if we need to change our code frequently in an experiment.

```
# python3
>>> from scapy.all import *
>>> a = IP()
>>> a.show()
####[ IP ]####
  version    = 4
  ihl        = None
  ...
```

(20pts) Task 1.1: Sniffing Packets

Wireshark is the most popular sniffing tool, and it is easy to use. We will use it throughout the entire lab. However, it is difficult to use Wireshark as a building block to construct other tools. We will use Scapy for that purpose. The objective of this task is to learn how to use Scapy to do packet sniffing in Python programs. A sample code is provided in the following:

```
#!/usr/bin/env python3
from scapy.all import *

def print_pkt(pkt):
    pkt.show()

pkt = sniff(iface='br-c93733e9f913', filter='icmp', prn=print_pkt)
```

The code above will sniff the packets on the `br-c93733e9f913` interface. Please read the instruction in the lab setup section regarding how to get the interface name. If we want to sniff on multiple interfaces, we can put all the interfaces in a list, and assign it to `iface`. See the following example:

```
iface=['br-c93733e9f913', 'enp0s3']
```

Task 1.1A. In the above program, for each captured packet, the callback function `print_pkt()` will be invoked; this function will print out some of the information about the packet. Run the program with the root privilege and demonstrate that you can indeed capture packets. After that, run the program again, but without using the root privilege; describe and explain your observations.

```
// Make the program executable
# chmod a+x sniffer.py

// Run the program with the root privilege
# sniffer.py

// Switch to the "seed" account, and
// run the program without the root privilege
# su seed
$ sniffer.py
```

Task 1.1B. Usually, when we sniff packets, we are only interested certain types of packets. We can do that by setting filters in sniffing. Scapy's filter use the BPF (Berkeley Packet Filter) syntax; you can find the

BPF manual from the Internet. Please set the following filters and demonstrate your sniffer program again (each filter should be set separately):

- Capture only the ICMP packet
- Capture any TCP packet that comes from a particular IP and with a destination port number 23.
- Capture packets comes from or to go to a particular subnet. You can pick any subnet, such as 128.230.0.0/16; you should not pick the subnet that your VM is attached to.

(10pts) Task 1.2: Spoofing MP Packets

As a packet spoofing tool, Scapy allows us to set the fields of IP packets to arbitrary values. The objective of this task is to spoof IP packets with an arbitrary source IP address. We will spoof ICMP echo request packets, and send them to another VM on the same network. We will use Wireshark to observe whether our request will be accepted by the receiver. If it is accepted, an echo reply packet will be sent to the spoofed IP address. The following code shows an example of how to spoof an ICMP packets.

```
>>> from scapy.all import *
>>> a = IP() ①
>>> a.dst = '10.0.2.3' ②
>>> b = ICMP() ③
>>> p = a/b ④
>>> send(p) ⑤
.
Sent 1 packets.
```

In the code above, Line ① creates an IP object from the IP class; a class attribute is defined for each IP header field. We can use `ls(a)` or `ls(IP)` to see all the attribute names/values. We can also use `a.show()` and `IP.show()` to do the same. Line ② shows how to set the destination IP address field. If a field is not set, a default value will be used.

```
>>> ls(a)
version      : BitField (4 bits)      = 4              (4)
ihl          : BitField (4 bits)      = None           (None)
tos          : XByteField              = 0              (0)
len          : ShortField              = None           (None)
id           : ShortField              = 1              (1)
flags        : FlagsField (3 bits)    = <Flag 0 ()>   (<Flag 0 ()>)
frag         : BitField (13 bits)     = 0              (0)
ttl          : ByteField               = 64             (64)
proto        : ByteEnumField          = 0              (0)
chksum       : XShortField             = None           (None)
src          : SourceIPField           = '127.0.0.1'   (None)
dst          : DestIPField             = '127.0.0.1'   (None)
options      : PacketListField        = []             ([])
```

Line ③ creates an ICMP object. The default type is echo request. In Line ④, we stack `a` and `b` together to form a new object. The `/` operator is overloaded by the IP class, so it no longer represents division; instead, it means adding `b` as the payload field of `a` and modifying the fields of `a` accordingly. As a result, we get a new object that represent an ICMP packet. We can now send out this packet using `send()` in Line ⑤. Please make any necessary change to the sample code, and then demonstrate that you can spoof an ICMP echo request packet with an arbitrary source IP address.

(10pts) Task 1.3: Traceroute

The objective of this task is to use Scapy to estimate the distance, in terms of number of routers, between your VM and a selected destination. This is basically what is implemented by the `traceroute` tool. In this task, we will write our own tool. The idea is quite straightforward: just send a packet (any type) to the destination, with its Time-To-Live (TTL) field set to 1 first. This packet will be dropped by the first router, which will send us an ICMP error message, telling us that the time-to-live has exceeded. That is how we get the IP address of the first router. We then increase our TTL field to 2, send out another packet, and get the IP address of the second router. We will repeat this procedure until our packet finally reach the destination. It should be noted that this experiment only gets an estimated result, because in theory, not all these packets take the same route (but in practice, they may within a short period of time). The code in the following shows one round in the procedure.

```
a = IP()
a.dst = '1.2.3.4'
a.ttl = 3
b = ICMP()
send(a/b)
```

If you are an experienced Python programmer, you can write your tool to perform the entire procedure automatically. If you are new to Python programming, you can do it by manually changing the TTL field in each round, and record the IP address based on your observation from Wireshark. Either way is acceptable, as long as you get the result.

Part 2 (70 pts: 40 Pts + 30 pts the OPTIONAL part)

Firewall Exploration Lab

Copyright © 2006 - 2021 by Wenliang Du.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. If you remix, transform, or build upon the material, this copyright notice must be left intact, or reproduced in a way that is reasonable to the medium in which the work is being re-published.

1 Overview

The learning objective of this lab is two-fold: learning how firewalls work, and setting up a simple firewall for a network. Students will first implement a simple stateless packet-filtering firewall, which inspects packets, and decides whether to drop or forward a packet based on firewall rules. Through this implementation task, students can get the basic ideas on how firewall works.

Actually, Linux already has a built-in firewall, also based on `netfilter`. This firewall is called `iptables`. Students will be given a simple network topology, and are asked to use `iptables` to set up firewall rules to protect the network. Students will also be exposed to several other interesting applications of `iptables`. This lab covers the following topics:

- Firewall
- Netfilter
- Loadable kernel module
- Using `iptables` to set up firewall rules
- Various applications of `iptables`

Readings and videos. Detailed coverage of firewalls can be found in the following:

- Chapter 17 of the SEED Book, *Computer & Internet Security: A Hands-on Approach*, 3rd Edition, by Wenliang Du. See details at <https://www.handsonsecurity.net>.
- Section 9 of the SEED Lecture, *Internet Security: A Hands-on Approach*, by Wenliang Du. See details at <https://www.handsonsecurity.net/video.html>.

Lab environment. This lab has been tested on the SEED Ubuntu 20.04 VM. You can download a pre-built image from the SEED website, and run the SEED VM on your own computer. However, most of the SEED labs can be conducted on the cloud, and you can follow our instruction to create a SEED VM on the cloud.

2 Environment Setup Using Containers

In this lab, we need to use multiple machines. Their setup is depicted in Figure 1. We will use containers to set up this lab environment.

2.1 Container Setup and Commands

Please download the `Labsetup.zip` file to your VM from the lab's website, unzip it, enter the `Labsetup` folder, and use the `docker-compose.yml` file to set up the lab environment. Detailed explanation of the content in this file and all the involved `Dockerfile` can be found from the user manual, which is linked

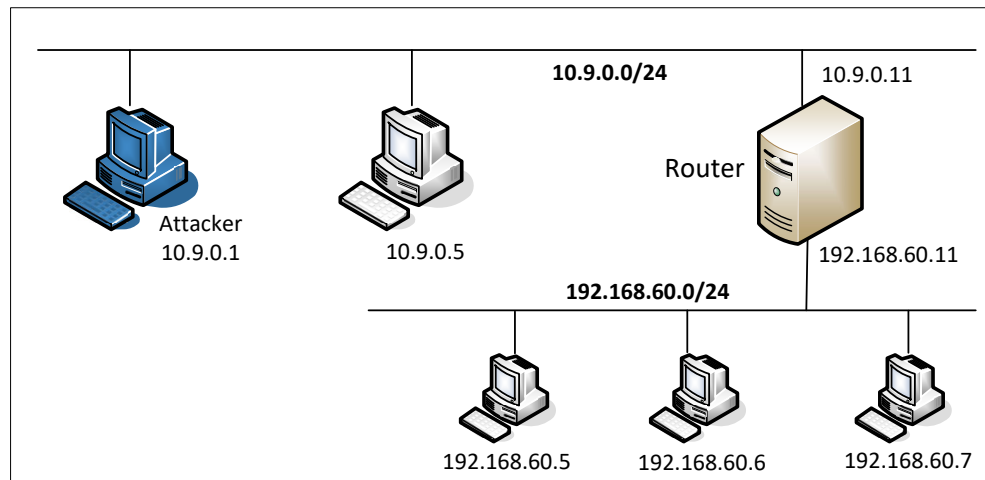


Figure 1: Lab setup

to the website of this lab. If this is the first time you set up a SEED lab environment using containers, it is very important that you read the user manual.

In the following, we list some of the commonly used commands related to Docker and Compose. Since we are going to use these commands very frequently, we have created aliases for them in the `.bashrc` file (in our provided SEEDUbuntu 20.04 VM).

```

$ docker-compose build # Build the container images
$ docker-compose up    # Start the containers
$ docker-compose down  # Shut down the containers

// Aliases for the Compose commands above
$ dcbuild              # Alias for: docker-compose build
$ dcup                 # Alias for: docker-compose up
$ dcdown               # Alias for: docker-compose down
  
```

All the containers will be running in the background. To run commands on a container, we often need to get a shell on that container. We first need to use the `"docker ps"` command to find out the ID of the container, and then use `"docker exec"` to start a shell on that container. We have created aliases for them in the `.bashrc` file.

```

$ dockps              // Alias for: docker ps --format "{{.ID}} {{.Names}}"
$ docksh <id>         // Alias for: docker exec -it <id> /bin/bash

// The following example shows how to get a shell inside hostC
$ dockps
b1004832e275  hostA-10.9.0.5
0af4ea7a3e2e  hostB-10.9.0.6
9652715c8e0a  hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#
  
```

// Note: If a docker command requires a container ID, you do not need to

4 Task 2: Experimenting with Stateless Firewall Rules

In the previous task, we had a chance to build a simple firewall using `netfilter`. Actually, Linux already has a built-in firewall, also based on `netfilter`. This firewall is called `iptables`. Technically, the kernel part implementation of the firewall is called `Xtables`, while `iptables` is a user-space program to configure the firewall. However, `iptables` is often used to refer to both the kernel-part implementation and the user-space program.

4.1 Background of `iptables`

In this task, we will use `iptables` to set up a firewall. The `iptables` firewall is designed not only to filter packets, but also to make changes to packets. To help manage these firewall rules for different purposes, `iptables` organizes all rules using a hierarchical structure: table, chain, and rules. There are several tables, each specifying the main purpose of the rules as shown in Table 1. For example, rules for packet filtering should be placed in the `filter` table, while rules for making changes to packets should be placed in the `nat` or `mangle` tables.

Each table contains several chains, each of which corresponds to a `netfilter` hook. Basically, each chain indicates where its rules are enforced. For example, rules on the `FORWARD` chain are enforced at the `NF_INET_FORWARD` hook, and rules on the `INPUT` chain are enforced at the `NF_INET_LOCAL_IN` hook.

Each chain contains a set of firewall rules that will be enforced. When we set up firewalls, we add rules to these chains. For example, if we would like to block all incoming `telnet` traffic, we would add a rule to the `INPUT` chain of the `filter` table. If we would like to redirect all incoming `telnet` traffic to a different port on a different host, basically doing port forwarding, we can add a rule to the `INPUT` chain of the `mangle` table, as we need to make changes to packets.

4.2 Using `iptables`

To add rules to the chains in each table, we use the `iptables` command, which is a quite powerful command. Students can find the manual of `iptables` by typing "`man iptables`" or easily find many tutorials from online. What makes `iptables` complicated is the many command-line arguments that we

Table 1: iptables Tables and Chains

Table	Chain	Functionality
filter	INPUT FORWARD OUTPUT	Packet filtering
nat	PREROUTING INPUT OUTPUT POSTROUTING	Modifying source or destination network addresses
mangle	PREROUTING INPUT FORWARD OUTPUT POSTROUTING	Packet content modification

need to provide when using the command. However, if we understand the structure of these command-line arguments, we will find out that the command is not that complicated.

In a typical `iptables` command, we add a rule to or remove a rule from one of the chains in one of the tables, so we need to specify a table name (the default is `filter`), a chain name, and an operation on the chain. After that, we specify the rule, which is basically a pattern that will be matched with each of the packets passing through. If there is a match, an action will be performed on this packet. The general structure of the command is depicted in the following:

```
iptables -t <table> -<operation> <chain> <rule> -j <target>
-----
Table          Chain          Rule          Action
```

The rule is the most complicated part of the `iptables` command. We will provide additional information later when we use specific rules. In the following, we list some commonly used commands:

```
// List all the rules in a table (without line number)
iptables -t nat -L -n

// List all the rules in a table (with line number)
iptables -t filter -L -n --line-numbers

// Delete rule No. 2 in the INPUT chain of the filter table
iptables -t filter -D INPUT 2

// Drop all the incoming packets that satisfy the <rule>
iptables -t filter -A INPUT <rule> -j DROP
```

Note. Docker relies on `iptables` to manage the networks it creates, so it adds many rules to the `nat` table. When we manipulate `iptables` rules, we should be careful not to remove Docker rules. For example, it will be quite dangerous to run the `"iptables -t nat -F"` command, because it removes all the rules in the `nat` table, including many of the Docker rules. That will cause trouble to Docker containers. Doing this for the `filter` table is fine, because Docker does not touch this table.

(10 pts) Task 2.A: Protecting the Router

In this task, we will set up rules to prevent outside machines from accessing the router machine, except ping. Please execute the following `iptables` command on the router container, and then try to access it from 10.9.0.5. (1) Can you ping the router? (2) Can you telnet into the router (a telnet server is running on all the containers; an account called `seed` was created on them with a password `dees`). Please report your observation and explain the purpose for each rule.

```
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT
iptables -P OUTPUT DROP      ← Set default rule for OUTPUT
iptables -P INPUT DROP      ← Set default rule for INPUT
```

Cleanup. Before moving on to the next task, please restore the `filter` table to its original state by running the following commands:

```
iptables -F
iptables -P OUTPUT ACCEPT
iptables -P INPUT ACCEPT
```

Another way to restore the states of all the tables is to restart the container. You can do it using the following command (you need to find the container's ID first):

```
$ docker restart <Container ID>
```

(10 pts) Task 2.B: Protecting the Internal Network

In this task, we will set up firewall rules on the router to protect the internal network 192.168.60.0/24. We need to use the `FORWARD` chain for this purpose.

The directions of packets in the `INPUT` and `OUTPUT` chains are clear: packets are either coming into (for `INPUT`) or going out (for `OUTPUT`). This is not true for the `FORWARD` chain, because it is bi-directional: packets going into the internal network or going out to the external network all go through this chain. To specify the direction, we can add the interface options using `"-i xyz"` (coming in from the `xyz` interface) and/or `"-o xyz"` (going out from the `xyz` interface). The interfaces for the internal and external networks are different. You can find out the interface names via the `"ip addr"` command.

In this task, we want to implement a firewall to protect the internal network. More specifically, we need to enforce the following restrictions on the ICMP traffic:

1. Outside hosts cannot ping internal hosts.
2. Outside hosts can ping the router.
3. Internal hosts can ping outside hosts.
4. All other packets between the internal and external networks should be blocked.

You will need to use the `"-p icmp"` options to specify the match options related to the ICMP protocol. You can run `"iptables -p icmp -h"` to find out all the ICMP match options. The following example drops the ICMP echo request.

```
iptables -A FORWARD -p icmp --icmp-type echo-request -j DROP
```

In your lab report, please include your rules and screenshots to demonstrate that your firewall works as expected. When you are done with this task, please remember to clean the table or restart the container before moving on to the next task.

(10 pts) Task 2.C: Protecting Internal Servers

In this task, we want to protect the TCP servers inside the internal network (192.168.60.0/24). More specifically, we would like to achieve the following objectives.

1. All the internal hosts run a telnet server (listening to port 23). Outside hosts can only access the telnet server on 192.168.60.5, not the other internal hosts.
2. Outside hosts cannot access other internal servers.
3. Internal hosts can access all the internal servers.
4. Internal hosts cannot access external servers.
5. In this task, the connection tracking mechanism is not allowed. It will be used in a later task.

You will need to use the "-p tcp" options to specify the match options related to the TCP protocol. You can run "iptables -p tcp -h" to find out all the TCP match options. The following example allows the TCP packets coming from the interface eth0 if their source port is 5000.

```
iptables -A FORWARD -i eth0 -p tcp --sport 5000 -j ACCEPT
```

When you are done with this task, please remember to clean the table or restart the container before moving on to the next task.

Task 3: Connection Tracking and Stateful Firewall

In the previous task, we have only set up stateless firewalls, which inspect each packet independently. However, packets are usually not independent; they may be part of a TCP connection, or they may be ICMP packets triggered by other packets. Treating them independently does not take into consideration the context of the packets, and can thus lead to inaccurate, unsafe, or complicated firewall rules. For example, if we would like to allow TCP packets to get into our network only if a connection was made first, we cannot achieve that easily using stateless packet filters, because when the firewall examines each individual TCP packet, it has no idea whether the packet belongs to an existing connection or not, unless the firewall maintains some state information for each connection. If it does that, it becomes a stateful firewall.

(10 pts) Task 3.A: Experiment with the Connection Tracking

To support stateful firewalls, we need to be able to track connections. This is achieved by the conntrack mechanism inside the kernel. In this task, we will conduct experiments related to this module, and get familiar with the connection tracking mechanism. In our experiment, we will check the connection tracking information on the router container. This can be done using the following command:

```
# conntrack -L
```

The goal of the task is to use a series of experiments to help students understand the connection concept in this tracking mechanism, especially for the ICMP and UDP protocols, because unlike TCP, they do not have connections. Please conduct the following experiments. For each experiment, please describe your observation, along with your explanation.

- ICMP experiment: Run the following command and check the connection tracking information on the router. Describe your observation. How long is the ICMP connection state be kept?

```
// On 10.9.0.5, send out ICMP packets
# ping 192.168.60.5
```

- UDP experiment: Run the following command and check the connection tracking information on the router. Describe your observation. How long is the UDP connection state be kept?

```
// On 192.168.60.5, start a netcat UDP server
# nc -lu 9090

// On 10.9.0.5, send out UDP packets
# nc -u 192.168.60.5 9090
<type something, then hit return>
```

- TCP experiment: Run the following command and check the connection tracking information on the router. Describe your observation. How long is the TCP connection state be kept?

```
// On 192.168.60.5, start a netcat TCP server
# nc -l 9090

// On 10.9.0.5, send out TCP packets
# nc 192.168.60.5 9090
<type something, then hit return>
```

(Optional 10pts) Task 3.B: Setting Up a Stateful Firewall

Now we are ready to set up firewall rules based on connections. In the following example, the "-m conntrack" option indicates that we are using the conntrack module, which is a very important module for iptables; it tracks connections, and iptables relies on the tracking information to build stateful firewalls. The --ctstate ESTABLISHED,RELATED indicates that whether a packet belongs to an ESTABLISHED or RELATED connection. The rule allows TCP packets belonging to an existing connection to pass through.

```
iptables -A FORWARD -p tcp -m conntrack \
        --ctstate ESTABLISHED,RELATED -j ACCEPT
```

The rule above does not cover the SYN packets, which do not belong to any established connection. Without it, we will not be able to create a connection in the first place. Therefore, we need to add a rule to accept incoming SYN packet:

```
iptables -A FORWARD -p tcp -i eth0 --dport 8080 --syn \
        -m conntrack --ctstate NEW -j ACCEPT
```

Finally, we will set the default policy on FORWARD to drop everything. This way, if a packet is not accepted by the two rules above, they will be dropped.

```
iptables -P FORWARD DROP
```

Please rewrite the firewall rules in Task 2.C, but this time, **we will add a rule allowing internal hosts to visit any external server** (this was not allowed in Task 2.C). After you write the rules using the connection

tracking mechanism, think about how to do it without using the connection tracking mechanism (you do not need to actually implement them). Based on these two sets of rules, compare these two different approaches, and explain the advantage and disadvantage of each approach. When you are done with this task, remember to clear all the rules.

(Optional 10 pts) Task 4: Limiting Network Traffic

In addition to blocking packets, we can also limit the number of packets that can pass through the firewall. This can be done using the `limit` module of `iptables`. In this task, we will use this module to limit how many packets from `10.9.0.5` are allowed to get into the internal network. You can use "`iptables -m limit -h`" to see the manual.

```
$ iptables -m limit -h
limit match options:
--limit avg                max average match rate: default 3/hour
                           [Packets per second unless followed by
                           /sec /minute /hour /day postfixes]
--limit-burst number       number to match in a burst, default 5
```

Please run the following commands on router, and then ping `192.168.60.5` from `10.9.0.5`. Describe your observation. Please conduct the experiment with and without the second rule, and then explain whether the second rule is needed or not, and why.

```
iptables -A FORWARD -s 10.9.0.5 -m limit \
    --limit 10/minute --limit-burst 5 -j ACCEPT

iptables -A FORWARD -s 10.9.0.5 -j DROP
```

(Optional 10pts) Task 5: Load Balancing

The `iptables` is very powerful. In addition to firewalls, it has many other applications. We will not be able to cover all its applications in this lab, but we will experimenting with one of the applications, load balancing. In this task, we will use it to load balance three UDP servers running in the internal network. Let's first start the server on each of the hosts: `192.168.60.5`, `192.168.60.6`, and `192.168.60.7` (the `-k` option indicates that the server can receive UDP datagrams from multiple hosts):

```
nc -luk 8080
```

We can use the `statistic` module to achieve load balancing. You can type the following command to get its manual. You can see there are two modes: `random` and `nth`. We will conduct experiments using both of them.

```
$ iptables -m statistic -h
statistic match options:
--mode mode                Match mode (random, nth)
random mode:
[!] --probability p        Probability
nth mode:
[!] --every n              Match every nth packet
--packet p                 Initial counter value (0 <= p <= n-1, default 0)
```

Using the `nth` mode (round-robin). On the router container, we set the following rule, which applies to all the UDP packets going to port 8080. The `nth` mode of the `statistic` module is used; it implements a round-robin load balancing policy: for every three packets, pick the packet 0 (i.e., the first one), change its destination IP address and port number to 192.168.60.5 and 8080, respectively. The modified packets will continue on its journey.

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode nth --every 3 --packet 0 \
-j DNAT --to-destination 192.168.60.5:8080
```

It should be noted that those packets that do not match the rule will continue on their journeys; they will not be modified or blocked. With this rule in place, if you send a UDP packet to the router's 8080 port, you will see that one out of three packets gets to 192.168.60.5.

```
// On 10.9.0.5
echo hello | nc -u 10.9.0.11 8080
<hit Ctrl-C>
```

Please add more rules to the router container, so all the three internal hosts get the equal number of packets. Please provide some explanation for the rules.

Using the `random` mode. Let's use a different mode to achieve the load balancing. The following rule will select a matching packet with the probability `P`. You need to replace `P` with a probability number.

```
iptables -t nat -A PREROUTING -p udp --dport 8080 \
-m statistic --mode random --probability P \
-j DNAT --to-destination 192.168.60.5:8080
```

Please use this mode to implement your load balancing rules, so each internal server get roughly the same amount of traffic (it may not be exactly the same, but should be close when the total number of packets is large). Please provide some explanation for the rules.

8 Submission and Demonstration

You need to submit a detailed lab report, code and description. Simply attaching code without any explanation will not receive credits. Quality of the report authoring, and its presentation will be part of the grading. Making a crisp and well formed report will be part of the grade.